Bug World
Simulator Protocol and Client Guide

Software Engineering Project

P. Baumann, Constructor University, Spring 2026

Client CLI Interface

Your client must accept the following command-line arguments:

```
client <world_map> <bug1> <bug2> [ticks_per_frame] [fps]
```

Argument	Required	Description	Default
world_map	Yes	Path to a .world map file	—
bug1	Yes	Path to a .bug file (Red team program)	—
bug2	Yes	Path to a .bug file (Black team program)	—
ticks_per_frame	No	Number of simulation ticks per display frame	50
fps	No	Target frames per second for display refresh	10

Examples:

```
# Minimal — uses default 50 ticks/frame, 10 fps
python3 client.py Worlds/tiny.world Bugs/simple.bug Bugs/simple.bug

# Custom tick rate and frame rate
python3 client.py Worlds/cross.world Bugs/beetle.bug Bugs/firefly.bug 20 5

# Fast forward (high ticks, low fps)
python3 client.py Worlds/sopra.world Bugs/beetle.bug Bugs/beetle.bug 200 2
```

What Your Client Must Do

1. Create two named pipes (FIFOs) in a temporary directory
2. Launch the sim binary as a subprocess with --cmd-pipe and --data-pipe arguments

3. Open the pipes in the correct order (see below)
4. Send commands (STEP N / FETCH / QUIT) over the command pipe
5. Read responses from the data pipe until the END marker
6. Parse and display the world state
7. Clean up pipes and terminate the simulator on exit

Pipe Protocol

Starting the Simulator

The simulator is launched as a subprocess with pipe paths provided as arguments:

```
./sim \
  --cmd-pipe /tmp/bugworld_XXXX/cmd.pipe \
  --data-pipe /tmp/bugworld_XXXX/data.pipe \
  <world_file> <red_bug_file> <black_bug_file>
```

Opening Pipes (Order Matters)

This ordering is critical to avoid deadlock.

The simulator opens pipes in this order:

8. Opens cmd-pipe for reading (blocks until a writer connects)
9. Opens data-pipe for writing (blocks until a reader connects)

Your client must open them in the matching order:

10. Open cmd-pipe for writing first
11. Open data-pipe for reading second

```
# Correct order — matches simulator's open sequence
cmd_fd  = os.open(cmd_path,  os.O_WRONLY)
data_fd = os.open(data_path, os.O_RDONLY | os.O_NONBLOCK)
```

Tip: Add a short sleep (100ms) after launching the simulator subprocess before opening pipes, to give it time to reach the blocking open() call.

Commands (Client → Simulator)

Commands are plain text, one per line, terminated by \n. Send them to the command pipe.

Command	Description

FETCH\n	Return the current world state without advancing
STEP N\n	Advance N simulation ticks, then return the state
QUIT\n	Shut down the simulator cleanly

- N must be a positive integer (minimum 1).
- After FETCH or STEP, the simulator writes a full state response to the data pipe.
- After QUIT, the simulator exits (return code 0). No response is sent.

Response Format (Simulator → Client)

Every response to FETCH or STEP is a multi-line text block on the data pipe, ending with an END marker:

```
CYCLE <cycle_number>
MAP <rows> <cols>
ROW <cell_chars...>
ROW <cell_chars...>
...
STATS <red_alive> <black_alive> <red_food> <black_food>
END
```

Line	Format	Description
CYCLE	CYCLE <int>	Current simulation cycle (starts at 0)
MAP	MAP <rows> <cols>	Grid dimensions (rows = x-size, cols = y-size)
ROW	ROW <space-separated chars>	One grid row. Odd rows have a leading extra space
STATS	STATS <int> <int> <int> <int>	Red alive, Black alive, Red nest food, Black nest food
END	END	End-of-response sentinel (not part of the state)

ROW Cell Characters

Char	Meaning
R	Red bug (no food)

r	Red bug carrying food
B	Black bug (no food)
b	Black bug carrying food
#	Rock (impassable)
.	Empty cell
+	Red team nest
-	Black team nest
1-9	Food (digit = quantity)

Hexagonal offset: Odd-indexed rows (y = 1, 3, 5, ...) have an extra leading space in the ROW line to represent the hexagonal stagger.

Response Example

After sending FETCH\n on a 10x10 world at cycle 0:

```
CYCLE 0
MAP 10 10
ROW # # # # # # # # # #
ROW  # 9 9 . . . . 3 3 #
ROW # 9 # . B B B B B #
ROW  # . # B B B B B B #
ROW # . . 5 B B B B B #
ROW  # R R R R R 5 . . #
ROW # R R R R R R # . #
ROW  # R R R R R . # 9 #
ROW # 3 3 . . . . 9 9 #
ROW  # # # # # # # # # #
STATS 16 16 0 0
END
```

Reading from the Data Pipe

The data pipe should be opened in non-blocking mode. Read chunks in a loop until you see a line containing exactly END.

World Map Format

World files (.world) define the hexagonal grid:

```
10        <- width (number of columns in x)
10        <- height (number of rows in y)
# # # # # # # # # #
 # 9 9 . . . . 3 3 #
# 9 # . - - - - - #
 # . # - - - - - - #
# . . 5 - - - - - #
 # + + + + + 5 . . #
# + + + + + + # . #
 # + + + + + . # 9 #
# 3 3 . . . . 9 9 #
 # # # # # # # # # #
```

- First line: grid width (x dimension)
- Second line: grid height (y dimension)
- Remaining lines: space-separated cell characters
- Odd rows are indented by one space (hexagonal offset)
- Cell characters: . (empty), # (rock), + (red nest), - (black nest), 1-9 (food)
- Bugs spawn on nest cells at simulation start (one bug per nest cell)